

AYOUB AKOUHAR

Khouribga, Morocco • 0682484311 • akouharayoub12@gmail.com
linkedin.com/in/ayoub-akouhar • github.com/ayouakouh • leetcode.com/u/ayouakouh

PROFESSIONAL SUMMARY

Software Engineer with comprehensive experience in UNIX/Linux environments, network programming, and low-level software architecture developed through intensive project-based training at 42 School. Proven track record building core systems-level infrastructure, including a custom HTTP web server, an IRC server, and a Bash-inspired shell using C/C++ and UNIX native system calls. Proficient in concurrent connection handling, process isolation, standard internet protocols, memory optimization, and algorithmic problem-solving.

TECHNICAL SKILLS

Languages:	C, C++, Bash, Python, SQL
Systems & Net:	UNIX/Linux, TCP/IP, Socket Programming, Non-blocking I/O, Multiplexing (poll/select/epoll), HTTP, IRC
Core Concepts:	Object-Oriented Programming (OOP), Event-Driven Architecture, Concurrency, Parsing & Lexing, Memory Management, Data Structures & Algorithms
Tools & DevOps:	Git, GitHub, GDB, Valgrind, Makefiles, Linux CLI, Shell Scripting

TECHNICAL PROJECTS

HTTP Web Server

C++ / Linux

Event-Driven HTTP Service

42 School Project

- Engineered a fully functional, compliant HTTP web server from scratch in C++ to process raw client requests and serve static infrastructure.
- Architected non-blocking network I/O multiplexing loops to handle numerous concurrent connections efficiently without thread exhaustion.
- Implemented structural HTTP protocol parsing for incoming headers, URI routing rules, request body verification, error handling, and structured response status codes.

IRC Server

C++ / Network Programming

Multi-Client Event-Driven TCP Chat Server

42 School Project

- Developed a robust, multi-client Internet Relay Chat (IRC) server complying with official RFC specifications using advanced socket programming.
- Managed explicit connection life cycles, text-based messaging protocols, and secure network buffers via `socket`, `bind`, `listen`, and `accept` calls.
- Leveraged `poll()` API to build high-performance, single-threaded connection scaling loops handling active client channels, authentications, and message routing.

MiniShell

C / UNIX System Calls

Bash-Inspired UNIX Command-Line Interpreter

42 School Project

- Authored a complete UNIX command execution engine in C with structural parsing, lexical analysis, and clean command validation loops.

- Managed core operating system interaction via system routines including `fork`, `execve`, `waitpid`, asynchronous signal handlers, and environment variables.
- Programmed complex pipeline sequences and file redirection operations through native file descriptor manipulations using `pipe` and `dup2`.

Cub3D

C / Graphics & Mathematics

Raycasting 3D Graphics Engine

42 School Project

- Designed a real-time pseudo-3D perspective engine inspired by Wolfenstein 3D utilizing the Digital Differential Analysis (DDA) raycasting algorithm.
- Programmed low-level mathematical projection calculations, texture mapping, camera plane manipulation, fluid player navigation, and strict wall collision arrays.

Systems & Networking Mini-Projects

C / Shell Utilities

Pipex, Minitalk, NetPractice, Fractol

42 School Core

- **NetPractice:** Mastered IP addressing structures, subnet routing tables, mask equations, and infrastructure troubleshooting methodologies.
- **Pipex & Minitalk:** Synthesized custom process inter-communication schemes using pipes and bitwise asynchronous signal handling (SIGUSR1/SIGUSR2).
- **Fractol:** Generated interactive deep-math visualizations of Mandelbrot and Julia fractal algorithms utilizing thread-bound rendering loops.

EDUCATION

42 Network Software Engineering Program

Khouribga, Morocco

Intensive Peer-to-Peer Curriculum (Systems Programming, Algorithms, & Networking)

2024 - Present

ALGORITHMIC PROBLEM SOLVING

LeetCode Contributor & Practitioner

leetcode.com/u/ayoakouh

- Consistently practicing data structures, runtime complexity minimization, and memory optimization paradigms to guarantee clean, performance-first algorithmic logic.